Security of Smart Environments - Summary

CHAPTER 1: IoT — Foundations, Threats, and Case Studies

What a Smart Environment Is, and Why Its Security Is Different

A smart environment is a physical space — a home, hospital, factory, city, or vehicle — instrumented with devices that bridge the digital and physical worlds. **Sensors** translate the physical world into data; **actuators** turn digital commands back into physical action. Together they let the environment perceive, reason, decide, and intervene, exhibiting context awareness, automation, and remote access.

The defining conceptual shift from ordinary IT security is that **the users live inside the system**. A breach of a web service harms information; a breach of a smart environment harms people. The governing lesson of the whole subject is that here **security failures become physical and privacy failures become personal**.

Virtually every such environment runs the same four-stage loop — **sense, connect, analyze, act** — and each stage is both an attack surface and a defensive opportunity. Three properties make the resulting security problem uniquely hard: the data is personal and continuous, the actuation is physical (so safety, not just security, is at stake), and the scale is enormous — billions of cheap, long-lived, rarely-patched devices.

The Building Blocks: IoT Devices and Layered Architecture

Smart environments are built on **IoT devices**, which are constrained, heterogeneous, and networked. The environment inherits every weakness of its component devices (default credentials, missing update mechanisms, exposed services) and then adds new attack surfaces on top — gateways, cloud back-ends, mobile apps, and AI pipelines. The security-relevant "thing" is usually an **embedded device**: a small, single-purpose computer with a CPU, non-volatile memory, connectivity, and a power source.

Devices sit in a broader **ecosystem** organized as a layered architecture, where every boundary between layers is simultaneously a point of attack and a place for defense:

Layer	Role	Security character
Users & applications	Phones, dashboards, voice assistants	Where people manage the environment
Cloud	Platform, storage, ML, third-party services	Richest target — concentrates data and capability

Layer	Role	Security character
Fog & network	Local servers, 5G edge	Latency-critical analytics near the data
Edge & gateway	Protocol translation, onboarding	Sees traffic in clear text at the moment of translation
Device	Sensors, actuators, firmware	Physically exposed and constrained — the root of vulnerability

The heterogeneity of devices is why the IoT relies on a sprawling **protocol landscape** rather than one unifying standard — spanning physical links, short-range wireless (Bluetooth/BLE, Zigbee, Z-Wave, NFC, RFID, Wi-Fi), cellular, and application messaging (MQTT, CoAP, HTTP, etc.), organized geographically into PAN, LAN, MAN, and WAN.

The State and History of IoT Security

Connected devices already outnumber humans, and most will never be patched. Security was historically absent by design: industrial and embedded systems assumed private networks, were migrated onto IP and then the open Internet, and relied on weak embedded passwords. Because security was bolted on reactively, insecure legacy devices remain in use and systems downgrade to insecure protocols for interoperability, while firmware updates stay hard or impossible.

The threat landscape evolved through **three ages**: the *Age of Exploration* (~2005–2009), dominated by security-by-obscurity and air-gapping; the *Age of Exploitation* (~2011–2019), when devices exploded, attacks were monetized, nation-states engaged, and technologies like TLS, secure boot, and TPMs appeared but were applied inconsistently; and the *Age of Protection* (2020–present), characterized by ubiquity, legislation, and security built into devices by design. Across all three, the **root causes never change**: weak or default credentials, unnecessary service exposure, absent updates, and vulnerable ecosystem interfaces — the same thread running through recent incidents (Verkada, CyberAv3ngers, Kia, BadBox, the Aisuru/Kimwolf botnets).

Case Study I: Stuxnet

Stuxnet is the pivotal case because it was the moment software crossed into destructive physical effect. Its goal was to **sabotage the Iranian nuclear program**, and it succeeded by making enrichment centrifuges run at damaging speeds while feeding operators normal-looking readings. It targeted **Industrial Control Systems** managed by Siemens Step7 PLCs, choosing to quietly degrade the program rather than destroy it outright — making it the widely regarded **opening act of cyber warfare**.

At a high level: suspected US/Israeli origin, signed with stolen certificates to appear legitimate, delivered across the **air gap** via USB drives carried by unwitting personnel. Its defining technical move was a **man-in-the-middle library substitution** — it replaced the DLL mediating Step7–PLC communication, so it could rewrite the controller's logic while

returning falsified, reassuring data to anyone inspecting the PLC. The engineers sensed the centrifuges were malfunctioning yet saw screens insisting all was normal, and their interventions had no effect — the precise embodiment of a security failure becoming a physical one. A cruder modern echo is the **Aliquippa water-utility attack**, where an Internet-exposed PLC protected only by a default password was hijacked with no exploit at all.

The Kia Portal Case: The Ecosystem as the Weakest Link

The Kia case teaches the complementary lesson: the weakest link is often not the device but the **ecosystem interface** around it. Researchers could locate, unlock, start, and track any recent Kia — and retrieve the owner's personal details — starting from only a license plate, by abusing the **dealer web portal** and its back-end API. The car itself was never compromised; a badly designed web application surrounding it was enough. This is **OWASP IoT risk #3, insecure ecosystem interfaces**.

Case Study II: Mirai and the Botnet Threat

Where Stuxnet was precision state sabotage, **Mirai** is the opposite pole: mass, indiscriminate compromise of cheap consumer devices for raw disruptive power. It turns Linux-based IoT devices (IP cameras, home routers) into **bots** controlled as a **botnet**. A bot is a secretly compromised, hard-to-trace machine (a "zombie"); a botnet is a coordinated collection of them, usable for DDoS, spam, sniffing, keylogging, click fraud, and more.

Mirai grows through a **self-amplifying loop**: each infected device scans for others, brute-forces a small dictionary of default credentials, reports successes to command-and-control, and enrolls new victims. Its most consequential deployment was the **2016 DDoS attack on the DNS provider Dyn**, which disrupted much of the Internet. Mirai operates in three stages — infect, protect itself (killing remote-access services; note it lives in volatile memory, so a reboot clears it but reinfection is immediate), and attack on command. Crucially, the attack traffic originates from thousands of innocent third-party devices, making DDoS both powerful and hard to trace.

Mirai remains dangerous because its **source code was publicly released**, turning it into a reusable "playbook" that spawned a family of variants. A decade later, descendants like **Aisuru and Kimwolf** set DDoS records above 30 Tb/s — proving the devices changed but the vulnerabilities did not. **BadBox 2.0** adds a further dimension: a factory-preinstalled backdoor (a **supply-chain compromise**, OWASP risk #5) that no user action can prevent, because the device ships already compromised and the update channel belongs to the attacker.

Securing the IoT

Securing the IoT is structurally hard: a vast attack surface, severely limited device resources, missing standards, entangled safety-and-security concerns, security not prioritized by price-competing manufacturers, scarce expertise, near-impossible field updates, and diffuse liability.

The foundational framework is the **CIA triad** — **Confidentiality** (plus privacy, since IoT data is personal, protected via anonymization, encryption, or perturbation), **Integrity** (readings and commands unaltered — the property Stuxnet subverted), and **Availability** (services accessible, and critically in *real time*, since a delayed brake or insulin command is a safety failure). Two further essentials are **identification/authentication** (of devices, applications, and users) and **trust** (established deliberately across layers and between components).

The authoritative catalogue of failures is the **OWASP Top 10 IoT risks**, which functions as the vocabulary for every incident in the chapter:

#	Risk
1	Weak, guessable, or hardcoded passwords
2	Insecure network services
3	Insecure ecosystem interfaces
4	Lack of secure update mechanism
5	Use of insecure or outdated components
6	Insufficient privacy protection
7	Insecure data transfer and storage
8	Lack of device management
9	Insecure default settings
10	Lack of physical hardening

Flaws divide into **design flaws** (architectural, often unfixable) and **library flaws** (a single bad shared component propagating into millions of devices). Threats can also be catalogued per architectural layer — sensing (node capture, false-data and code injection, replay), network (routing attacks like sinkhole and wormhole, spoofing, MITM, DoS/DDoS), middleware (SQL injection, signature wrapping, cloud malware), gateways (insecure onboarding, re-encryption exposure), and application (data theft, access-control bypass, reprogramming). Defensively, the same small set of disciplined measures recurs: disallow weak passwords and add lockout/MFA, minimize exposed ports and interfaces, encrypt data in every state, enable updates, minimize collected data, and harden devices physically — applied consistently across the device's entire life, from secure onboarding to secure decommissioning.

CHAPTER 2: Attestation and Collective Remote Attestation

The Attestation Problem

Attestation answers a simple but hard question: how can one party be confident a remote device is running exactly the software it should be, and nothing else? A compromised device can behave normally while exfiltrating data or manipulating readings. Attestation belongs at the lowest (perception) layer of IoT security, among the physical, resource-constrained devices whose internal state is in question.

Remote attestation is a challenge-response protocol between a **challenger** (verifier) and a **prover** (the device under scrutiny). An honest prover must be able to produce a token convincing the verifier it is in the expected state; a compromised prover must be unable to forge an acceptable token. Reasoning about it requires a precise **adversary model**, an escalating ladder of attacker power: software-only attackers (remote or local), physical non-intrusive attackers (observing secrets, side channels), stealthy physical-intrusive attackers (extracting secrets and cloning), and fully physical-intrusive attackers (modifying hardware). Two capabilities cut across all of these: full control of the **communication** channel and **mobility** (erasing traces of presence). Schemes are characterized by how far up this ladder they can defend.

Software-Based Attestation

The simplest approach is **software-based attestation**, requiring no special hardware, exemplified by **SWATT**. It assumes a software-only attacker and relies on a challenge-response in which the device computes a checksum over its own memory that the verifier (holding an exact replica of the expected state) can predict. Its security rests on an **argument about overhead**: a compromised device that keeps a clean copy of the original code and redirects attestation reads to it must pay a small but measurable time penalty, which a verifier measuring response time can detect. The verification is engineered to make this overhead unavoidable — memory is traversed in an unpredictable, exhaustive, pseudo-random order seeded by a fresh nonce, and the computation is non-parallelizable with no useful precomputation or replay.

Its limitations are decisive and motivate everything after it: the whole argument holds **only against software-only attackers**, it demands a strict and precisely-known time bound, and it **fails over multi-hop networks** where network latency swamps the timing signal.

A related concern is the **TOCTTOU attack** (time-of-check-to-time-of-use): verifying memory contents does not guarantee that those verified contents are what actually *runs*, since malware can swap code in the gap between check and execution. **SCUBA** addresses secure *recovery* of an already-infected device, built on **ICE (Indisputable Code Execution)**, which binds verification and execution into a single atomic, uninterruptible, self-checksumming operation so no substitution window exists. Self-checksumming schemes must resist both **checksum-forgery attacks** (faking a correct value) and **speed-based attacks** (buying back the time cheating costs, e.g. via optimization, colluding devices, or a faster proxy), each neutralized by a specific design countermeasure.

Hardware-Based Attestation

Hardware-based attestation replaces the fragile timing foundation with a tamper-resistant hardware component — canonically a **TPM (Trusted Platform Module)** — that holds secrets untrusted software can never reach and signs attestation tokens the software cannot forge. Using a digital signature rather than a MAC makes results **publicly verifiable** by anyone holding the public key.

Any attestation must terminate in a **root of trust** assumed trustworthy by construction. A **static root of trust** is established at boot: **secure boot** verifies code signatures against an immutable ROM-stored bootloader/public key, and the TPM's **PCRs (Platform Configuration Registers)** accumulate a tamper-evident, ordered summary of the boot sequence via the "extend" operation. A **dynamic root of trust** (e.g. Intel TXT) instead establishes a fresh trusted measurement of the system's *current* state at any moment without rebooting.

The limitation is cost: full tamper-resistant hardware on every one-dollar sensor is economically infeasible, which drives the central question answered by **SMART** — what is the *minimum* hardware needed for secure attestation? SMART adds only a small set of features (attestation code in ROM, secure key storage reachable only from that ROM code, access-control logic, and reset/erasure on violation), delivering a hardware-anchored dynamic root of trust on cheap microcontrollers.

Hybrid Attestation

Hybrid attestation is the dominant paradigm for constrained devices: combine a minimal set of trusted hardware features (exclusive key access, immutability, uninterruptible execution) with a software routine doing the measurement. **TyTAN** extends this to devices running multiple mutually-distrusting applications at once, using an **Execution-Aware Memory Protection Unit** to grant secrets only to specific trusted code, and providing secure task loading, secure inter-process communication, secure storage, and real-time guarantees — notably allowing *interruptible* measurement to honor real-time deadlines.

Interactive attestation harbors a **denial-of-service vulnerability**: an attacker can flood a device with challenges, forcing it to abandon real work. **SeED** inverts the initiative — the *device* self-triggers attestation on a tamper-proof, pseudo-random schedule that both parties can reproduce — so external flooding accomplishes nothing. This **non-interactive** design also resists mobile adversaries, because the attacker cannot predict the next attestation time to clean up beforehand.

Collective Remote Attestation (Swarm Attestation)

Single-prover attestation does not scale: a verifier polling a million devices one by one faces linear cost and prohibitive time, and fitting trusted hardware to every node is infeasible. **Collective Remote Attestation (CRA)** verifies an entire network more efficiently by having devices **attest one another**, so each device becomes the verifier of a neighbor and the

work spreads across the network. A key benefit is that all exchanges are **one-hop**, which sidesteps the multi-hop timing problem.

The family develops through successive protocols, each fixing a predecessor's flaw:

- **SEDA** — the foundational scheme. Organizes devices into a **spanning tree**; each node attests its children and aggregates results upward to the root, achieving logarithmic scaling with cheap symmetric crypto. Its flaws: all devices must participate, results are not publicly verifiable, and — fatally — a physically compromised internal node can falsify its entire subtree's verdict.
- **SANA** — heals SEDA's wounds using an **aggregate signature** scheme that lets many devices' proofs be merged into one compact, publicly verifiable signature. Only provers need a Trusted Execution Environment; a compromised aggregator can combine but cannot *forge* signatures, so it cannot rewrite its subtree's verdict. Verifier cost scales with the (small) number of *bad* devices, not the network size.
- **DARPA** — adds **continuous liveness monitoring** between attestations. Since physical tampering requires removing a device, devices broadcast heartbeats; a node that disappears is flagged even if it later attests correctly. It inherits SEDA's tree-based limitations and consumes power.
- **LISA, SCAPI** — further spanning-tree refinements (LISA introduces the Quality-of-Swarm-Attestation metric; SCAPI improves resiliency and handles physical attacks), all sharing the family's weakness: **unsuitability for highly dynamic networks**.

That shared weakness motivates **dynamic CRA** for mobile swarms (drones, robots) whose neighborhoods change constantly. **PADS** abandons the tree for **consensus via gossip**: every device self-attests, then opportunistically exchanges findings with whatever neighbors are in range, using **maximum consensus** (equivalent to a bitwise-OR that monotonically accumulates evidence of compromise and never erases it). It uses a three-state encoding (good / bad / **unknown**) and lets the verifier read a run-time network snapshot from any attested device. Its drawback is membership rigidity: a fixed-length per-device bitmask means adding a node requires reconfiguring all devices. **SALAD** targets the same regime via broadcast with strong cryptography, at higher overhead. The most refined scheme, **Bloom-filter-based CRA**, keeps PADS's gossip-and-consensus philosophy but replaces the rigid bitmask with a **Bloom filter** — a probabilistic, space-efficient membership structure that is naturally monotonic (false positives possible, false negatives impossible). This dissolves PADS's membership rigidity, lets devices join and leave fluidly, and propagates faster — at the cost of relying on a shared symmetric key, probabilistic (inexact) identification of compromised devices, and uncertainty about how many nodes participated.

CHAPTER 3: IoT Authentication

Why Authentication in the IoT Is Special

Authentication convinces a verifier that a party is who it claims to be — classically by proving knowledge of a stored secret. The IoT breaks this in two ways: **physical exposure** (an unattended device can be captured and its stored key extracted, after which the attacker impersonates it perfectly, since to the verifier the key *is* the identity) and **energy** (cryptographic operations drain battery-powered nodes). These pressures motivate building authentication, and even key generation, on **physical properties** of the radio link and hardware that an attacker cannot easily read out or reproduce.

Physical Layer Security (PLS)

PLS exploits three physical facts: the wireless channel is random but **reciprocal** (A→B looks like B→A over a short window); it **decorrelates over about half a wavelength** (~6 cm at 2.4 GHz), so an eavesdropper even a short distance away sees a different channel; and **every transmitter is physically unique**, carrying analog manufacturing imperfections that act as a hardware fingerprint. From these flow two uses: **authentication** (asking whether a frame comes from the same place or radio as trusted frames) and **key generation** (two parties measuring the same reciprocal channel extract matching secret bits without transmitting them). The threat model assumes the attacker is farther than half a wavelength away and cannot copy the hardware.

Authentication comes in two flavors: **channel-based** (comparing the channel of consecutive frames — verifies location continuity but cannot authenticate the very first frame, which needs another anchor) and **hardware-based / RF fingerprinting** (identifying the radio by features like carrier frequency offset, IQ imbalance, transient shape, and amplifier non-linearity — increasingly via deep learning on raw IQ samples; works even if the device moves). Both reduce to a **threshold decision** trading false alarms against missed detections, increasingly tuned adaptively with machine learning.

Key generation from channel reciprocity runs a four-stage pipeline — probing, quantization, information reconciliation (error-correcting the mismatches), and privacy amplification (hashing out any leaked information). PLS has real limits: static environments yield few random bits, a nearby attacker sees a correlated channel, the channel can be manipulated, and RF fingerprints can be imitated with a software-defined radio or replayed. The honest conclusion is that **PLS complements cryptography; it does not replace it**. The four main PLS authentication approaches (consecutive-frame comparison, third-party relay, channel-derived keys, and cryptographic tags) each trade cost against the breadth of attacks they withstand — there is no free authentication.

Physically Unclonable Functions and Fuzzy Extractors

A **Physically Unclonable Function (PUF)** deliberately exploits nano-scale manufacturing variations: a circuit whose behavior depends on physical randomness no manufacturer can control or reproduce (e.g. the start-up pattern of SRAM cells). Its decisive property is **unclonability** — the secret is not stored and therefore cannot be read out; it exists only as a physical behavior evoked on demand. A PUF is operated as a **challenge–response** function:

at **enrollment** a server records challenge–response pairs; at **authentication** it sends a challenge and checks the response, proving the device possesses its original physical fingerprint without any stored key.

PUF responses are **noisy** — tolerable for comparison-style authentication but fatal for key generation, which needs an *exactly* reproducible key. A **fuzzy extractor** bridges this with two procedures: **Gen** (run once at enrollment, producing a strong key plus public **helper data**) and **Rep** (run on every later use, reconstructing the identical key from a noisy reading plus the helper data). The canonical **code-offset construction** uses an error-correcting code to cancel the noise, and the helper data can be stored or transmitted in the clear because it leaks almost nothing about the key.

PUFs divide into **weak PUFs** (few challenge–response pairs — used as a key store, e.g. SRAM) and **strong PUFs** (an astronomically large CRP space — suitable for authentication with a fresh challenge every session, e.g. the arbiter PUF). Strong PUFs face **modeling attacks**: because the challenge-to-response mapping is often mathematically regular, machine learning can learn an accurate model from a few thousand observed pairs and then predict any response without the device — making non-linearity and obfuscation central to PUF design.

Lightweight Multi-Factor Mutual Authentication

These primitives combine into a complete **lightweight multi-factor mutual authentication protocol**, built around a PUF for hardware identity and a fuzzy extractor for stable key recovery. The design philosophy is **asymmetry**: the powerful **gateway** (the authentication anchor that enrolls each chip and stores its secrets) does the heavy lifting, while the constrained device performs only cheap hashes and XORs. A three-message exchange achieves mutual authentication and agrees a shared session key, under a pessimistic Dolev–Yao attacker who fully controls the channel. Three recurring features carry the security: the device only hashes and XORs, every message carries a fresh timestamp and nonce, and the long-term secret never travels on the wire (appearing only inside hash arguments). A **dynamic key derivation scheme** then expands the single session key into a continually refreshed stream of per-message keys for confidentiality and integrity. The security analysis delivers the full suite of properties — indistinguishability, anonymity, MITM and replay resistance, masquerade resistance, and forward secrecy — from just two primitives: a one-way hash and freshness.

Standardized Onboarding and Session Protocols

Real deployments run on standards. **Matter** (smart-home) commissions a device through three layered steps that map to the three levels of trust: a factory-burnt **Device Attestation Certificate** proving genuineness (manufacturer level), a **password-authenticated key exchange** from a printed code proving physical possession (user level), and a **Node Operational Certificate** enabling autonomous device-to-device sessions thereafter (autonomous level). **LoRaWAN** onboards via **Over-The-Air Activation**, a two-message join handshake deriving session keys from a provisioned root key — pedagogically valuable for its known weaknesses (shared root keys across a product line, replayable nonces, static-key

ABP devices, and no device-to-device authentication). **EDHOC + OSCORE** is the IETF's lightweight answer for constrained nodes that cannot afford full TLS: EDHOC is a compact three-message handshake (mutual authentication, forward secrecy) and OSCORE protects application-layer messages end-to-end, working *through* proxies and gateways. **Bluetooth Low Energy pairing** is a case study in how device I/O capabilities dictate security: legacy "Just Works" pairing offers no protection, modern LE Secure Connections uses ECDH but still leaves "Just Works" open to man-in-the-middle, so a screenless device is pushed toward the weakest model.

Before any protocol can run, a device must *acquire* a trustworthy identity. Frameworks like **IEEE 802.1AR** (factory IDevID + owner-issued LDevID), **DICE** (binding identity to the hash of the running code, so tampered firmware loses its keys), and **FIDO Device Onboard** (zero-touch onboarding via a supply-chain voucher) all share one principle: **factory identity first, owner-issued operational identity second**. The whole subject divides into **user-to-device** authentication (passwords, MFA, passkeys, OAuth device grant, physical presence, behavioral biometrics) and **device-to-device** authentication (symmetric keys, PUFs, certificates — demanding scale and resilience to key compromise). The sharpest closing observation is that the weakest point is the **delegation boundary** — the moment a human grants a machine authority to act autonomously.

CHAPTER 4: Secure Computation

The Meaning and Limits of Privacy

Privacy is the protection of an individual's personal information across its whole lifecycle — collection, use, retention, disclosure, and disposal — and is usefully understood as a promise of freedom from harm. Defining it precisely is hard. A naive definition ("the analyst learns nothing new about Alice") fails in the presence of **auxiliary information**: an analysis revealing that smoking correlates with lung cancer can harm a known smoker *even if her data was never in the dataset*, and forbidding such learning would forbid science itself. The workable refinement shifts the baseline to **"with Alice versus without Alice"**: an analysis is private if its result is almost unchanged whether or not Alice's record is included — protecting her individual contribution, not the population-level knowledge. This is the seed of differential privacy.

A critical consequence is that **removing identifiers does not guarantee privacy**. Three real episodes make the point: the **GIC incident** (anonymized medical records re-identified by linking the quasi-identifier *age + sex + zip code* against public voter rolls); the **AOL release** (pseudonymous search histories that were self-identifying in their content); and the **Netflix** de-anonymization (6–8 approximate ratings sufficed to re-identify users via public IMDb reviews). The unifying lesson: naive anonymization fails against an adversary with auxiliary information, which motivates cryptographically grounded tools — beginning with secure computation.

Multi-Party Computation

Multi-Party Computation (MPC) lets mutually distrustful parties jointly compute a function of their private inputs while revealing nothing beyond the output. The foundational guarantee is that no party learns more than before, *except what the output itself unavoidably implies*. Canonical illustrations: the **Millionaires' Problem** (two parties learn who is richer without revealing their wealth — a concrete protocol using public-key encryption shows this is ordinary cryptography, not magic); **coin tossing** (two distrustful parties agree on a fair random bit — **Blum's protocol** forces one party to *commit* before the other reveals, so neither can bias the result); and **secure dating** (a private logical AND revealing a match only when interest is mutual, protecting the rejected party).

Every protocol's guarantee is meaningful only against a stated **adversary model**, in increasing strength: **semi-honest** ("honest-but-curious" — follows the protocol but tries to infer from what it sees), **malicious** (deviates arbitrarily), and **covert** (will cheat only if it can avoid being caught — deterred by a high detection probability).

Homomorphic Encryption

Homomorphic encryption (HE) allows computation on data while it stays encrypted. The property emerges from a scheme's algebra: **RSA is multiplicatively homomorphic** (the product of ciphertexts decrypts to the product of plaintexts), while other schemes — most importantly **Paillier** — are **additively homomorphic** (combining ciphertexts yields the encryption of the *sum*). For a long time these were separate worlds: one homomorphism or the other, never both. Additive HE supports adding two secrets and multiplying a secret by a *known constant*, which suffices for all **linear operations** (scalar products with public vectors, FIR filtering, linear transforms). Non-linear functions require an interactive technique called **blinding**: one party masks the encrypted value with a random affine transform, a second party (holding the key) operates on the harmlessly randomized plaintext, and the first party unmasks — the "blind-operate-unblind" pattern (illustrated by squaring an encrypted number).

Fully Homomorphic Encryption (FHE) achieves the grander ambition: *both* unbounded addition and multiplication, non-interactively. This matters because addition and multiplication mod 2 are exactly **XOR and AND**, which are Turing-complete — a scheme that evaluates both can compute *any* function on encrypted data. FHE was cryptography's "holy grail" until **Gentry's 2008 breakthrough**. A simple integer-based scheme (secure under the **approximate-GCD assumption**, where a deliberate **noise** term defeats a GCD attack) exhibits the central obstacle: addition grows noise additively, but **multiplication squares it**, and once noise exceeds a threshold decryption fails — yielding only a **somewhat-homomorphic** scheme (many additions, few multiplications). The key idea that crosses to "fully" is **bootstrapping**: homomorphically evaluating the scheme's *own decryption circuit* on a noisy ciphertext (using a published *encryption of the secret key*) to produce a fresh ciphertext of the same value with the noise reset — a "valve" that drains noise before overflow. **Gentry's theorem**: if a scheme can evaluate its own decryption circuit, it can evaluate

everything. The cost, however, is enormous — keys and ciphertexts balloon into megabytes and gigabytes — so making FHE practical remains the field's central engineering challenge.

Garbled Circuits

Yao's Garbled Circuits is an earlier approach to general two-party computation that encrypts the *circuit* itself. A function is expressed as a Boolean circuit; the **garbler** assigns each wire two random, unlabeled keys (one per bit value) and replaces each gate with an **encrypted, permuted truth table** where only the row matching the actual input keys can be decrypted. The **evaluator** propagates one opaque key per wire from inputs to outputs, learning nothing about intermediate values because keys carry no visible mark of which bit they encode. A final **decryption table** translates output keys to bits.

One piece is missing: how the evaluator obtains the key for *her own* private input without revealing it and without receiving both keys. This is **oblivious transfer (OT)** — a 1-out-of-2 protocol (buildable from additively homomorphic encryption) in which the receiver gets exactly the chosen value while the sender learns nothing about the choice. Naively, garbled circuits are correct but expensive; three classic optimizations make them practical: **point-and-permute** (the evaluator jumps straight to the one decryptable row), **high-speed encryption** (replacing double encryption with a single hash-based mask), and **free XOR** (making XOR gates cost essentially nothing via a global offset, so designers build circuits to minimize costly AND gates). Standard building blocks (adders, comparators, multiplexers, minimum-finders) are assembled this way, and much of the work — garbling and even OT — can be shifted into an **offline precomputation** phase so the online phase is fast.

Hybrid Protocols

HE and GC have **complementary profiles**: HE is communication-cheap but computation-expensive and excels at linear arithmetic; GC is computation-cheap but communication-expensive and excels at non-linear operations (comparison, minimum selection, division). A **hybrid protocol** uses each where it is cheapest — linear stretches in HE, non-linear stages in GC — stitched together by a **blinding-based interface** that converts a value from one encrypted representation to the other.

Secret Sharing

Secret sharing is a third, structurally different paradigm: split a secret into **shares**, one per party, such that shares individually look random but authorized subsets reconstruct the secret. Its original motivation was safeguarding keys (replication trades loss risk against exposure risk; sharing resolves the dilemma). A **threshold scheme** makes any $t+1$ shares sufficient while t or fewer reveal nothing — giving resilience against both loss and exposure. A crucial property is that **privacy is not incremental**: an adversary collecting shares one at a time learns *nothing* until the threshold, at which point knowledge jumps abruptly from none to full.

Constructions: the trivial cases (one share = no privacy; all-shares-required = easy but rigid) bound the useful middle. A geometric attempt (points on a circle) fails because two shares already confine the center to a line, leaking partial information. The working idea is **interpolation codes**: a degree-t polynomial is uniquely determined by $t+1$ points but *completely* undetermined by fewer, leaking nothing. This is **Shamir's scheme** — the secret is the polynomial's constant term, shares are evaluations at public points, and reconstruction uses **Lagrange interpolation**. The guarantees are expressible formally in information-theoretic terms (via entropy, access structures, and adversary structures), and a fundamental bound states that **each share must be at least as large as the secret**. A vivid application is **secret image sharing**, where a mandatory **permutation pre-step** destroys spatial correlations so the shadow images look like pure noise rather than leaking the picture's outline. Secret sharing is also the basis for **MPC**: parties share their inputs, compute directly on the shares (additions locally, multiplications with interaction), and selectively **open** each output to whichever audience should receive it — never exposing the inputs.

Statistical Approaches to Privacy

A complementary family concerns *publishing* a dataset usefully while resisting re-identification. **k-anonymity** requires each record to be indistinguishable from at least $k-1$ others on its **quasi-identifiers**, achieved by **generalization** (coarsening values) and **suppression** (removing outliers). Its weakness is that it ignores the sensitive attribute: the **homogeneity attack** (every record in a class shares the same sensitive value) and the **background-knowledge attack** defeat it. **ℓ-diversity** patches this by requiring each class to contain at least ℓ "well-represented" sensitive values (strengthened by an *entropy* variant), but it falls to the **skewness attack** (ignoring the global distribution) and the **similarity attack** (distinct-but-semantically-similar values). **t-closeness** patches further by requiring each class's sensitive distribution to be close to the global one, measured with **Earth Mover's Distance** (which captures semantic distance). This patch-and-respond cycle — each refinement closing one attack — suggests no syntactic condition on a table will ever be definitive.

Differential privacy breaks the cycle by shifting the guarantee from the *shape of the table* to the *behavior of the release mechanism.* Two datasets are **neighbors** if they differ in one individual's record; a randomized mechanism is **ε-differentially private** if neighboring datasets produce output distributions within a multiplicative factor e^{ϵ} everywhere. This directly realizes the "with-Alice-versus-without-Alice" ideal and holds against *any* adversary with *any* background knowledge — immune by construction to the earlier attacks. A deterministic exact release can never satisfy it, so the mechanism **adds calibrated noise**: the required noise scales with the query's **global sensitivity** (the worst-case influence of one individual) divided by the **privacy budget ε**. **Laplace noise** is used because its multiplicative tails match the multiplicative guarantee exactly. ε is the explicit, tunable dial trading privacy against accuracy — the first approach to offer a provable, attack-independent guarantee.

CHAPTER 5: Computing with Private Data

Why Cryptography Alone Is Not Enough

Conventional cryptography protects data **at rest and in transit**, but to compute on it with standard tools one must decrypt it — and at that instant the protection vanishes. **Signal processing in the encrypted domain (s.p.e.d.)** closes this gap: performing a meaningful computation (a comparison, a distance, a classification) directly on data that stays encrypted, so the party doing the work never sees the input and often not even the output. Real deployments remain rare, not for lack of tools but because generic secure computation is dramatically slower and more bandwidth-hungry than its plaintext equivalent. Practicality therefore comes from **tailoring the protocol to the specific application**, invoking the expensive cryptographic machinery as little as possible.

This is an engineering discipline governed by a three-way tension between **accuracy, privacy, and performance**, proceeding through four phases: application-domain analysis (understand the plaintext problem), protocol definition (fix accuracy, privacy, and complexity requirements), **protocol optimization** (the creative core — reshaping the algorithm, choosing which inputs enter the computation, and choosing how they are represented in bits), and testing. The phases loop: test results send the designer back to re-optimize.

Privacy-Preserving Biometric Recognition

Biometrics are the archetypal application, since a template is irreplaceable (a leaked fingerprint cannot be reissued). Two tasks differ structurally: **authentication** (one-to-one — check a single claimed template, output yes/no) and **identification** (one-to-many — compare the query against *every* template via a bank of distance computations plus a secure minimum-selection stage, far more expensive and a prime optimization target). Three optimization levers recur:

- **Input representation** — fingerprint matching by **minutiae** is accurate but has irregular, conditional logic that resists secure implementation, whereas the **FingerCode** representation reduces matching to a fixed-length Euclidean distance (clean, regular arithmetic) at a small accuracy cost. Choosing the representation that linearizes the matching function is often the single most important decision.
- **Algorithmic optimization** — in iris recognition, replacing each user's private hiding mask with a single **common mask** makes the mask *public* (and therefore free to apply in a garbled circuit), cutting gate count sharply for under 1% accuracy loss and large latency/bandwidth gains.
- **Feature-vector length and bit-size** — jointly traded against accuracy; no configuration is universally best, so the designer picks the point on the accuracy–complexity surface matching the binding resource constraint.

Secure ECG Diagnosis

Remote diagnosis is a **two-sided privacy problem**: the patient must protect a sensitive ECG, and the service must protect its proprietary classifier. The solution encrypts the signal, classifies it in the encrypted domain, and returns an encrypted verdict only the patient can read — neither party seeing the other's secret. The plaintext baseline (**Ge et al.**) extracts autoregressive features, applies a quadratic discriminant function, and walks a **decision tree** to assign one of six cardiac states. The decision stage is formalized as a **Linear Branching Program (LBP)** — a decision tree whose branches test the *sign of a linear combination* of inputs.

Two architectures evaluate the LBP securely. The **full-garbled-circuit** solution compiles everything into one circuit, but the multiplications in each scalar product cause a *quadratic* gate blow-up and prohibitive communication. The **hybrid (HE + GC)** protocol fixes this by routing the linear scalar products to homomorphic encryption and leaving only the sign tests and tree traversal to a small garbled circuit, bridged by a **blinding interface** that additively shares each discriminant between the parties. Making this practical requires **quantization** (mapping floating-point features to integers — larger scaling buys accuracy but costs bits) and two independent accelerators: shrinking the **bit-length** (a worst-case analysis gives a safe bound, but experiment shows far fewer bits actually suffice) and **feature reduction** (dropping a feature for a couple of accuracy points, which also lowers the bit-length needed). Head-to-head, the hybrid wins on communication by two-to-three orders of magnitude, is comparable or better on computation at small security parameters, but can *lose* to pure GC at large security parameters where HE arithmetic becomes expensive — the recurring moral that no single primitive dominates.

An alternative classifier is a **neural network**, attractive because it is built almost entirely from weighted sums and a single repeated non-linearity — operations secure computation handles well. The non-linearity is the crux: the smooth **tansig** is transcendental and hard to garble, so it is replaced by **satlins** (a piecewise-linear saturating approximation evaluable with a few gates), with **arg max** (a sequence of comparisons) selecting the class. Comparing the two classifiers, the hybrid LBP wins on communication, the full-GC neural network wins on computation (no public-key arithmetic) and on raw accuracy — again, the choice depends on which resource binds.

ODIN: Privacy-Preserving Consensus for Information Fusion

The final application is genuinely distributed. In the IoT, many sensing devices must combine measurements into one collective conclusion — **information fusion** — with no central server, using a **consensus algorithm** where each node repeatedly exchanges and averages its estimate with neighbors until the network agrees. Standard consensus leaks each node's private value to its neighbors every round. **ODIN** is a privacy-preserving consensus protocol that lets non-trusting nodes cooperate while no node learns another's measurement, using the same divide-and-assign philosophy as the ECG work. The repeated **update steps** are protected by **modular additive blinding** (efficient masking where each node's value is always masked by randomness chosen by *another* node) together with **proxy re-encryption** — a primitive

letting a semi-trusted proxy translate a ciphertext from one party's key to another's without learning the message, which makes each update's cost **independent of how many neighbors a node has**. The single final **decision step** (comparing the converged estimate against a threshold) is handled by a **garbled circuit**, the non-arithmetic operation at which circuits excel.

CHAPTER 6: ML Security

Why Machine Learning Belongs in Cybersecurity

Security has shifted from deterministic detection toward statistical inference because the volume of network events far exceeds what manual analysis can handle. Machine learning offers three decisive advantages: it **automatically discovers** useful information from massive datasets; it **generalizes**, learning the underlying structure of malicious behavior so it detects both known and *unknown* attacks; and it provides **operational speed**, cutting through alert noise to surface what merits human attention. This replaced the older **signature-based** approach — exact, predefined patterns that catch only what they have a literal description of and are evaded by trivial modification. ML is instead **probabilistic**, judging an activity malicious with a degree of belief, which grants flexibility and generalization at the cost of perfect certainty.

The Taxonomy of ML Algorithms

Algorithms organize along two axes. **Shallow vs. deep**: shallow models require human **feature engineering** but are simple enough to run anywhere, including constrained IoT devices; deep models **autonomously select features** from raw data but demand heavy compute and data. **Supervised vs. unsupervised**: supervised learns a mapping from *labeled* examples to predefined classes; unsupervised discovers structure in *unlabeled* data on its own terms.

The principal **supervised shallow** algorithms: **Naive Bayes** (applies Bayes' theorem, assuming features are independent — "naive" but often effective); **logistic regression** (fits a bounded sigmoid so outputs are interpretable as class probabilities); **support vector machines** (find the maximum-margin separating hyperplane defined by the closest points, the *support vectors*, and use the **kernel trick** to handle non-linearly-separable data by lifting it into a higher dimension); **random forest** (an ensemble combining many decision trees via **bagging**, voting by majority); **K-nearest neighbour** (classify by majority vote among the K closest labeled points); and **shallow neural networks** (layers of neurons that each multiply inputs by weights, sum, add a bias, and apply a non-linear activation). The **unsupervised shallow** tasks are **clustering** (grouping similar items, e.g. k-means) and **association** (finding frequent if-then patterns judged by *support* and *confidence*, as in market-basket analysis).

The **deep architectures**: **deep neural networks** (many hidden layers building a representational hierarchy); **convolutional neural networks** (convolution and pooling layers specialized for spatial data like images); and **recurrent neural networks** (a feedback

connection giving memory for sequential data). The **unsupervised deep** architectures are **deep belief networks** (stacked restricted Boltzmann machines trained greedily) and **stacked autoencoders** (an hourglass network forced to compress input through a narrow bottleneck, learning an efficient encoding).

Evaluation and Operational Realities

Every binary evaluation starts from the **confusion matrix** (true/false positives and negatives), from which four metrics derive: **accuracy** (overall correctness), **precision** (penalizes false alarms), **recall** (penalizes missed detections), and **F1-score** (their harmonic mean). In security, **accuracy is treacherous** because of class imbalance — with attacks rare among benign traffic, a trivial "everything benign" classifier scores above 99% while catching nothing — so F1, precision, and recall are the operative metrics. Several counterintuitive lessons follow: **shallow can beat deep** when it fits the data better; **specialized detectors** (one per attack type) far outperform a single general detector; fair comparison across published methods is genuinely hard; and the two error types have asymmetric costs — **false positives** drain analyst time while **false negatives** can compromise an organization outright. Models also **decay** as the threat landscape shifts, requiring continuous, labor-intensive **re-training**.

ML-based intrusion detection is organized by data granularity — **packet** (finest, content-rich but context-poor), **flow** (aggregated, context-rich but content-poor), **session** (complete interactions, but huge volumes), and **log** (application-level records, rich but format-heterogeneous) — each with characteristic trade-offs and preferred models. Beyond IDS, ML supports **continuous authentication** (behavioral biometrics) and **steganalysis**. Finally, ML is also a **weapon**: **ML-based attacks** induce **information leakage**, with stealing targets spanning user-activity data, model information (architecture, parameters, training data), and authentication secrets.

CHAPTER 7: ML for APT

Critical Infrastructures and Advanced Persistent Threats

Critical infrastructures (water, energy, telecom, finance, healthcare) demand **continuous availability** and are **morphologically different** from ordinary networks — heterogeneous assemblies spanning IT and industrial control equipment, organized into corporate, supervisory, and field layers separated by an **air gap**. Threats span two inversely related axes, sophistication and prevalence: from prevalent, unsophisticated **commodity** crime up through **targeted attacks**, **hacktivists**, **industrial espionage**, to the rare, highly capable **nation-state** operations — which correspond to **Advanced Persistent Threats**.

An **APT** (per NIST) is an adversary with sophisticated expertise and significant resources operating through multiple attack vectors: **Advanced** (custom tooling, zero-days), **Persistent**

(remaining covertly inside for months or years), **Threat** (organized, well-financed, goal-driven). Their objectives are to steal information, spy over time, or sabotage, and they concentrate on high-value targets (government, defense, media). A key distinction is **actor vs. tooling vs. campaign**: the APT is a *group*, which *uses malware* within *campaigns*. Because a group **reuses code**, its malware carries a signature of authorship — the basis for **attribution**.

Triage and Classifier Design

Malware analysis in this setting is governed by a **medical-triage** analogy: analysts are a scarce resource, so samples must be **prioritized**, **false positives minimized** (each wastes analyst time), and screening must be **fast** — hence reliance on **static analysis** (examining a sample without running it) feeding ML **classifiers**. Features are drawn from the Windows Portable Executable structure, with **imported API functions** by far the richest signal.

A conceptual obstacle is the **not-belonging class problem**: APT samples are rare while "normal malware" is an unbounded, ill-defined category impossible to represent faithfully. The remedy is to avoid modeling it — train **only on positive APT examples**. Two designs follow: a **single random forest** scoring authorship (accurate but with high *retraining* cost since adding a group requires retraining everything), and a **set of one-class classifiers** (one per APT, giving modular, incremental retraining). Evaluated on the **dAPTaset** benchmark, both detect and attribute near-perfectly; the decisive difference is training time, where the modular one-class design wins by orders of magnitude.

A structural weakness remains: performing detection and identification in a **single phase** makes detection a side-effect of attribution, producing **false negatives** (a genuine APT sample matching no known group is discarded). The **multi-step solution** separates the tasks — a dedicated binary detector first, then per-group identifiers whose negative class deliberately includes the detector's false positives. This yields a **three-outcome** verdict: screened-out non-APT, identified APT, and crucially a **detected-but-unattributed** category (a possible new group) that single-phase designs could not represent. Across these pipelines, **random forest** consistently offers the best combination of accuracy and speed.

API-Based and Image-Based Attribution

A distinct line builds on API-usage fingerprints. **ApiScout** recovers API usage from memory dumps (dynamic, slow, manual), and **ApiVector** encodes that usage as a compact bit vector for comparison. **APTScout** replaces the slow dynamic front-end with **fast, automated static recovery** using the Ghidra reverse-engineering framework — easy for **static imports** (read from the Import Table) but requiring a clever **bottom-up strategy** for **dynamic imports** (searching decompiled code for `GetProcAddress` calls and working backward to the library and function name, since malware hides APIs by resolving them at runtime). Samples are compared by a **weighted Jaccard similarity** (weighting rare, discriminative APIs more, e.g. by entropy) and attributed via **K-nearest neighbour**, confirming that same-group malware really does share enough API structure for purely static attribution.

A third, visual approach **turns a binary into a grayscale image** (bytes → pixel intensities), exploiting that same-family malware looks alike in **layout and texture**. The framework extracts **opcodes** (robust to superficial variation), renders them as an image, normalizes size with **bicubic interpolation**, and extracts **Histogram of Oriented Gradients** features (capturing the distinguishing edges and bands) for classification. Because APT malware is far rarer than ordinary malware, the data is imbalanced, addressed by **resampling** (under-, over-, and SMOTE-family combined methods). The recurring result is that **random forest** robustly outperforms a multi-layer perceptron, often doing well even without resampling.

CHAPTER 8: Adversarial Machine Learning

Why Classifiers Are Vulnerable

Machine learning **assumes a benign world** — training and test data from the same distribution, noise independent of the model — and nothing in standard training anticipates an adversary. **Adversarial perturbations** exploit this: a perturbation so small the image looks unchanged to a human flips the classifier's decision. The key insight is that **random noise is not enough** — only a *specific, tailored* perturbation, aimed in the right direction, flips the label. The extreme is the **one-pixel attack** (changing a single pixel, often with high confidence). Attacks reach the **physical world** too — patterned eyeglasses defeating face recognition, stickers making a stop sign read as a speed limit, a 3D-printed "turtle" classified as a rifle from many angles — though physical perturbations must survive camera capture and are more conspicuous. This reframes the field from **"ML for security" to "security of ML."**

The Threat Framework

Attacks classify along three axes: **influence** (*causative*, altering training data, vs. *exploratory*, only probing the deployed model), **security violation** (the **CIA triad** restated: *integrity*, *availability*, *confidentiality/model-stealth*), and **specification** (*targeted*, forcing a specific wrong class, vs. *indiscriminate*, any error). Geometrically, a sample near a decision boundary is barely moved by random noise (almost all directions keep it correct) but is flipped by a small step aimed *across* the boundary. The **curse of dimensionality** makes this worse: for a linear classifier, random noise contributes zero on average while a tailored perturbation aligned with the weights contributes an effect **proportional to the number of features** — so high-dimensional classifiers are *more* exposed. Attacks are **white-box** (model known, optimal perturbation computed directly) or **black-box** (probing by queries, exploiting **transferability** — a perturbation crafted on a surrogate model often works on the target). Attackers also exploit **empty regions** of feature space, where models assign arbitrary labels to inputs that resemble nothing real.

Poisoning acts at training time, inserting mislabeled points near the boundary so it bends toward the attacker's advantage, making later evasion easier. Both poisoning and adversarial-example attacks have developed catalogues of named methods (**FGSM**, **BIM**, **JSMA**, **Carlini & Wagner**, **universal perturbations**), tracing a progression from one-step to iterative, from whole-image to few-pixel, from single-model to transferable, from white-box to black-box, and from lab to real world. The natural defense is **adversary-aware retraining** (reshaping the boundary with boundary-adjacent samples), but robust deep-learning defense remains open because of **opacity** — we often cannot say which internal features a network uses, making **interpretability (explainable AI)** a prerequisite for defense.

Backdoor Attacks

A **backdoor** poisons training so the model behaves normally on ordinary inputs but exhibits an attacker-chosen behavior whenever a secret **trigger** is present (e.g. any image with a small yellow star classified as a dog). It works by causing specific neurons to become **trigger detectors** that override the normal decision only when activated. Under **full control** of training the attacker labels the trigger explicitly; under the harder **partial control** the attacker contributes a small fraction of corrupted samples and must also be stealthy at training time. A good backdoor is stealthy, reliable, hard to remove, **unobtrusive** (no impact on normal inputs), and **robust** (surviving recapture, compression). Triggers are classified by visibility, locality, and adaptivity, and backdoors extend to the physical and video domains.

The subtlest form is the **clean-label attack** — every poisoned image keeps its *correct* label, so inspecting labels reveals nothing. The basic version is weak because the trigger becomes "sufficient but not necessary" (the network relies on genuine features instead). The fix is a **good bait**: craft poisoned target-class images that are hard to classify from their own content, forcing the network to latch onto the trigger. **Feature-transfer** attacks go further — in transfer learning, an **adversarial instance** (looking like one class but with deep features of another) can achieve a **one-shot kill** (the "poison frogs" construction) with a single poisoned sample. **Defenses** act at training time (**within-class activation clustering** — poisoned samples split off into a distinct cluster) or test time (**reverse-engineering the trigger** since a backdoored class is anomalously easy to drive into; **saliency-map examination**), and **removal** exploits that backdoor neurons are *dormant on benign inputs* — **fine-pruning** removes the inactive neurons, eliminating the backdoor before benign accuracy degrades. Finally, the same mechanism has **beneficial uses**: **DNN watermarking** (a backdoor becomes an ownership proof) and **honeypots** (a planted trigger lures attackers into a detectable trap).

CHAPTER 9: Private ML

Machine-Learning-as-a-Service exposes a client's data to the server. **Private ML** lets the server operate on data it cannot read, splitting into **cryptographic** approaches (computing on

encrypted values) and **differential privacy** (noise hiding any single record).

Cryptographic Private ML

CryptoNets targets secure **inference** outsourcing: the client sends encrypted inputs, the server applies a pre-trained network, and returns an encrypted prediction only the client can decrypt. The **model is public** (plaintext weights) while the data is private, keeping operations cheap, using **leveled BGV FHE** (bounded multiplicative depth, avoiding bootstrapping) and **SIMD batching** (one ciphertext packs many instances, amortizing cost). Because FHE supports only addition and multiplication, network elements are re-expressed: weighted sums are used directly, max-pooling is avoided (non-polynomial), mean-pooling is scaled to remove division, and sigmoid/ReLU are **replaced by polynomial approximations** — notably the **square activation** $f(x)=x^2$. The square layers dominate the cost, yet the network reaches 99% on MNIST with no meaningful accuracy loss, and batching yields high throughput despite high fixed latency.

Private learning tackles the harder problem of outsourcing *training*, keeping all inputs, weights, gradients, and intermediate values encrypted against an honest-but-curious server. This requires **full** (not leveled) FHE because training's multiplicative depth is unbounded, the sigmoid is approximated by a truncated **Taylor polynomial**, and **key switching (bootstrapping)** is applied after every product to refresh noise — the client decrypting and re-encrypting each round to reset ciphertexts. Accuracy lands within two points of the plaintext model, recoverable further by retaining more Taylor terms (at higher cost).

Private multi-party learning generalizes to many clients jointly training one model over the union of their data, with neither the server nor other clients seeing anyone's raw data. It uses an **additively homomorphic** scheme (LWE) — sufficient because the server only needs to *sum* gradients, never multiply — in a federated-style protocol where the global weights live on the server **only in encrypted form**. Additive homomorphism is far cheaper than full FHE: ciphertext addition costs microseconds versus milliseconds for encryption, so the server's aggregation is nearly free and cost sits with the participants.

Differential-Privacy-Based ML

Cryptography protects data *during* computation but a trained model can still **leak its training data through its outputs**. The relevant attacks are **reconstruction** (features → raw data), **model inversion** (model → representative features), and **membership inference** (determining whether a specific record was in the training set, exploiting a model's higher confidence on data it was trained on). **Differential privacy** defends by bounding any single record's influence through calibrated noise, injected at one of three points in gradient descent: **objective perturbation** (noise in the loss function), **gradient perturbation** (noise in each update step, the basis of DP-SGD), or **output perturbation** (noise in the final parameters). The earlier noise enters, the more optimization reshapes it; the later, the more directly it controls the released model. The **privacy budget ε** is the explicit dial — smaller ε means

stronger privacy and typically lower accuracy — and in practice all three perturbation points are used across both low- and high-dimensional problems.